AI Systems Interview Portfolio

Cohort C · Interview Preparation Phase (IRP)

New Age Software Engineering Program · 8 Sessions · 2 Hours Each · Gemini Free Tier

8	2 Hours	Portfolio Modules	Gemini 1.5 Flash	Free Tier	8 Python Modules
Sessions	Duration per Session	Format	LLM	Cost	Deliverable

Target Audience

Students who completed the 6–7 month NASE program covering Python, GenAI, prompt engineering, RAG, LLMOps, agents, and LangGraph. They use AI coding tools (Antigravity). No web dev background required — all deliverables are Python scripts or notebooks. Only requirement: free Gemini API key from aistudio.google.com.

Program End Goal

By the end of 8 sessions, students will have built a portfolio of 8 standalone AI engineering modules — covering structured output, LLMOps, serverless patterns, RAG, RAG evaluation, agent routing, vision/OCR, and system design — all using Gemini 1.5 Flash (free tier) and local tools (sentence-transformers, ChromaDB). Students will be able to explain every module, its trade-offs, production limitations, and design decisions confidently in AI engineering interviews. Zero cost: only a free Google AI Studio API key is required.

Session-by-Session Breakdown

Session 1 **Tame the Chaos: Build a Structured Output Engine**

- System prompts with JSON schema definitions
- response_mime_type="application/json"
- Temperature = 0 for deterministic output
- Free-text vs. structured output comparison
- Token count logging

Deliverable: structured_output_engine.py + output_examples.json

Session 2

Watch Every Call: Build Your LLM Observatory

- log_llm_call() wrapper pattern
- Latency measurement with time.time()
- Token + cost estimation
- Manual quality scoring (1–5)
- CSV + JSON flat file output
- print_report() pass/fail stats

Deliverable: llm_logger.py + llm_logs.csv + eval_summary.json

Session 3

Think Serverless: Wrap AI in a Cloud-Ready Function

- handler(event, context) pattern
- Input validation + structured JSON response
- Cold start concept
- Cost-per-invocation thinking
- Environment variable management with python-dotenv

Deliverable: ai_handler.py + .env.example

Session 4

Build the Memory: Engineer a RAG Pipeline from Scratch

- Embeddings with sentence-transformers (all-MiniLM-L6-v2, local)
- Paragraph-level chunking
- ChromaDB persistent local collection
- Top-3 chunk retrieval
- Grounded generation vs. vanilla LLM comparison

Deliverable: rag_pipeline.py + chroma_db/ (local persistent)

Session 5

Grade the Brain: Evaluate and Improve Your RAG System

- Groundedness and faithfulness concepts
- Keyword-overlap relevance scoring
- Per-question tracking (query, retrieved chunks, answer, expected, scores)
- Before/after comparison with one changed parameter
- RAG failure mode articulation

Deliverable: rag_evaluator.py + rag_eval_report.csv

Session 6

Route with Intent: Build an Agent That Picks Its Tools

- Agent vs. chatbot distinction
- Intent classification with Gemini (structured JSON)
- 4 tool functions: rag_answer(), summarize_text(), safety_check(), ask_clarification()
- route(query) orchestrator
- Reasoning trace: Intent → Tool → Result

Deliverable: agent_router.py

Session 7 — See and Understand: Give Your AI Eyes with Vision/OCR

- Vision-language model vs. traditional OCR
- Multimodal prompting with PIL.Image
- Structured JSON extraction: extracted_text, document_type, key_fields, confidence_notes
- Human verification importance
- Same API key, same model, no extra cost

Deliverable: vision_ocr_module.py + sample_image.png + ocr_output.json

Session 8 — Tell the Story: System Design, Demo, and Interview Ready

- Portfolio README and Mermaid architecture diagrams
- Interview vocabulary: latency, throughput, cost, hallucination rate
- Production trade-off articulation per module
- Safety topics: prompt injection, PII, hallucination
- 2-minute spoken demo structuring

Deliverable: README.md + architecture_diagram.md + demo_script.md + viva_prep.md + module_summary.md

Quick-Reference Summary Table

#	Session Title	Key Topics	Deliverable
1	**Tame the Chaos: Build a Structured Output Engine**	System prompts with JSON schema definitions · response_mime_type="application/json" · Temperature = 0 for deterministic output…	structured_output_engine.py + output_examples.json
2	**Watch Every Call: Build Your LLM Observatory**	log_llm_call() wrapper pattern · Latency measurement with time.time() · Token + cost estimation…	llm_logger.py + llm_logs.csv + eval_summary.json
3	**Think Serverless: Wrap AI in a Cloud-Ready Function**	handler(event, context) pattern · Input validation + structured JSON response · Cold start concept…	ai_handler.py + .env.example
4	**Build the Memory: Engineer a RAG Pipeline from Scratch**	Embeddings with sentence-transformers (all-MiniLM-L6-v2, local) · Paragraph-level chunking · ChromaDB persistent local collection…	rag_pipeline.py + chroma_db/ (local persistent)
5	**Grade the Brain: Evaluate and Improve Your RAG System**	Groundedness and faithfulness concepts · Keyword-overlap relevance scoring · Per-question tracking (query, retrieved chunks, answer, expected, scores)…	rag_evaluator.py + rag_eval_report.csv
6	**Route with Intent: Build an Agent That Picks Its Tools**	Agent vs. chatbot distinction · Intent classification with Gemini (structured JSON) · 4 tool functions: rag_answer(), summarize_text(), safety_check(), ask_clarification()…	agent_router.py
7	**See and Understand: Give Your AI Eyes with Vision/OCR**	Vision-language model vs. traditional OCR · Multimodal prompting with PIL.Image · Structured JSON extraction: extracted_text, document_type, key_fields, confidence_notes…	vision_ocr_module.py + sample_image.png + ocr_output.json
8	**Tell the Story: System Design, Demo, and Interview Ready**	Portfolio README and Mermaid architecture diagrams · Interview vocabulary: latency, throughput, cost, hallucination rate · Production trade-off articulation per module…	README.md + architecture_diagram.md + demo_script.md + viva_prep.md + module_summary.md